

ПРАКТИЧЕСКАЯ РАБОТА № 9

Тема: Скрытый канал для передачи данных ICMP Tunnel.

Программное обеспечение:

1. ОС Kali Linux <https://www.kali.org/get-kali/#kali-platforms>
2. ОС Windows Server <https://www.comss.ru/download/page.php?id=9610>
3. Wireshark и PuTTY
<https://drive.google.com/drive/folders/1bruYgL3m6tOdxqZHoigMdIxeCoZurXek?usp=sharing>

Вопросы для обсуждения:

1. Что такое скрытый канал передачи данных?
2. Какой протокол используется в ICMP-туннелировании?
3. В чём заключается идея использования ICMP как средства передачи данных в обход сетевых ограничений?
4. Какие типы ICMP-пакетов чаще всего используются для создания туннелей?
5. Чем отличаются ICMP-запросы типа Echo Request и Echo Reply?
6. Почему ICMP-трафик часто игнорируется фаерволами и системами IDS/IPS?
7. Как можно использовать ICMP для обхода сетевых политик (например, запрета на использование SSH или VPN)?
8. Перечислите известные вам инструменты или утилиты для создания ICMP-туннелей.
9. Какие риски безопасности связаны с применением ICMP-туннелей?
10. Как можно обнаружить использование ICMP-туннеля в сети?

Цель: Получение теоретических и практических навыков построения и управления скрытым каналом для передачи данных ICMP Tunnel.

Основные теоретические сведения

1. Скрытый канал

Скрытый канал — это коммуникационный канал, пересылающий информацию методом, который изначально был для этого не предназначен.

Впервые понятие скрытого канала было введено в работе Батлера Лэмпсона «A Note of the Confinement Problem» 10 октября 1973 года, как «[каналы], не предназначенные для передачи информации совершенно, такие как воздействие служебной программы на загрузку системы». Чаще всего скрытый канал является паразитом по отношению к основному каналу: скрытый канал уменьшает пропускную способность основного канала. Сторонние наблюдатели обычно не могут обнаружить, что помимо основного канала передачи данных есть ещё дополнительный. Только отправитель и получатель знают это.

Скрытый канал носит своё название в силу того факта, что он спрятан от систем разграничения доступа даже безопасных операционных систем, так как он не использует законные механизмы передачи, такие как чтение и запись, и потому не может быть обнаружен или проконтролирован аппаратными механизмами обеспечения безопасности, которые лежат в основе защищённых операционных систем. В реальных системах скрытый канал практически невозможно установить, и также его часто можно обнаружить с помощью наблюдения за быстродействием системы; кроме того, недостатками скрытых каналов являются низкое отношение сигнал/шум и низкие скорости передачи данных (порядка нескольких бит в секунду). Их также можно удалить с защищённых систем вручную с высокой степенью надёжности, если воспользоваться признанными стратегиями анализа скрытых каналов.

Скрытые каналы могут проходить сквозь защищённые операционные системы, и необходимы особые меры для их контроля. Единственным проверенным методом контроля скрытых каналов является так называемый анализ скрытых каналов. В то же время, защищённые операционные системы могут предотвратить незаконное использование легальных каналов. Часто анализ легальных каналов на предмет скрытых объектов неверно

представляют как единственную успешную меру против незаконного использования легальных каналов. Поскольку на практике это означает необходимость анализировать большое количество программного обеспечения, ещё в 1972 было показано, что подобные меры неэффективны. Не зная этого, многие верят в то, что подобный анализ может помочь справиться с рисками, связанными с легальными каналами.

2. Туннелирование HTTP

Большая часть трафика приходит в Интернет после исследования, и сетевые администраторы предпринимают шаги по ограничению трафика своих сетей, но есть один порт, который никто не хочет закрывать - обычно это порт 80. Пользователи (и администраторы) постоянно просматривают веб сайты, независимо от того, нужно это для работы или нет. Жизненная основа присутствия компании в Интернет, так или иначе, требует наличие веб ресурса, а это требует наличия веб сервера, размещенного у хостинг провайдера или в сети компании. С каждым новым червем, ошибкой или уязвимостью, обнаруженной в IIS (Internet Information Services) или Apache, сетевые администраторы и администраторы безопасности пытаются заблокировать эти сервисы на маршрутизаторе или брандмауэре. Для идентификации атак многие используют IDS и IPS.

Туннелирование не представляет собой ничего нового. IPSec это, возможно, самый широко известный способ туннелирования, наиболее близким продолжением которого является SSH туннелирование. Однако, если атакующий не имеет достаточно высоких шансов на успех, маловероятно, что он выберет IPSec или SSH в качестве возможного порта проникновения. Не каждая компания предоставляет удаленный доступ через IPSec или SSH, но, если она делает это, сервер вероятно уже укреплен.

Из-за повсеместного распространения веб серверов они являются превосходной целью для атакующего. Кроме того, при условии, что они находятся за брандмауэром, веб сервера представляют из себя единственный,

но важный вход в целевую сеть. Это тот случай, где применяется туннелирование. Так как возросло понимание необходимости защиты, компании установили дополнительные системы защиты между веб сервером и Интернет, так же, как и непосредственно на самом веб сервере. Средства обнаружения вторжений (IDS) и Средства предотвращения вторжений (IPS) используются для идентификации и уведомления администраторов об обнаружении атаки против системы или сети. Но даже у них есть свои недостатки, и HTTP туннелирование может использоваться для обхода этих средств.

HTTP туннелирование использует клиентское приложение, чтобы инкапсулировать трафик в пределах HTTP заголовка. Затем трафик идет на сервер на другом конце канала связи, который обрабатывает пакет, "раздевает" заголовки HTTP пакета и переадресовывает пакет его заключительному адресату. И UDP и TCP трафик может использоваться для инкапсулирования. Это возможно из-за самой природы туннелирования, подобной IPSec туннелю, где реальным пакетом является полезная нагрузка исходного пакета.

В настоящее время есть только два приложения для туннелирования HTTP. Одно из них распространяется с открытым исходным кодом, другое коммерческий продукт. Первое приложение, доступное на сайте <http://www.nocrew.org/software/httpunnel.html>, это GNU HTTP tunnel. Второе - HTTP Tunnel корпорации HTTP-Tunnel. Оба они разработаны для обеспечения соединения к различным портам, осуществляя связь через 80 TCP порт.

Туннелирование HTTP может использоваться для доступа к портам, которые обычно недоступны из сети (Рис.1). Маршрутизатор имеет следующие правила:

```
inbound:

    permit tcp any host WWW port 80
    permit tcp any host WWW port 443
    permit tcp any host DNS/SMTP port 25
    permit udp any host DNS/SMTP port 53

outbound:

    permit ip any any
```

Политика безопасности брандмауэра:

```
inbound:

    permit ip host DNS/SMTP host SSH eq 22
    permit ip host DNS/SMTP host SSH eq 80
    permit ip host DNS/SMTP host SSH eq 443

outbound:

    permit ip any any
```

Рисунок 1 - Пример установки HTTP Tunnel

Атакующий взламывает веб сервер с установленным IIS. Не важно какой именно эксплойт он использует, важно то, что он способен эксплойтировать сервер и получить доступ к системе через командную строку. Как только он получает этот доступ, он загружает скомпилированную версию сервера HTTP Tunnel, hts. Синтаксис запуска сервера HTTP Tunnel следующий (Рис.2):

```
hts.exe -F (SRC PORT) (TARGET):(DST PORT)
```

Рисунок 2 – Синтаксис запуска сервера HTTP Tunnel

(SRC PORT) это порт, который будет переадресован, (TARGET) это IP адрес целевого хоста, а (DST PORT) это целевой порт, который будет принимать переадресовываемый трафик. Когда клиент посылает трафик на (SRC PORT) сервер автоматически переадресовывает его на (DST PORT) хоста (TARGET). (TARGET) может быть IP адресом системы, на которой запущен hts сервер. После установки hts сервера атакующий запускает клиент на своей системе и направляет его трафик на (SRC PORT) системы с установленным hts сервером. Синтаксис запуска клиента HTTP Tunnel такой же, как и для сервера (Рис. 3):

```
htc -F <SRC PORT> <TARGET>:<DST PORT>
```

Рисунок 3 – Синтаксис запуска клиента HTTP Tunnel

Клиент принимает трафик с и переадресовывает его на хосте.

Атакующий устанавливает hts на WWW хост и запускает htc на своей системе. Процесс hts слушает порт 80 WWW хоста и может переадресовывать трафик.

Когда атакующий соединяется с портом 1025 на своей системе, процесс htc инкапсулирует трафик в HTTP заголовки и переадресовывает его на веб сервер. Этот сервер принимает трафик, "раздевает" HTTP заголовок и переправляет трафик на 23 порт DNS/SMTP сервера. Таким образом, в то время как порт telnet на DNS/SMTP сервера напрямую недоступен вне корпоративного маршрутизатора, атакующий имеет доступ к этому порту, и может попробовать перебором узнать логин и пароль для доступа к системе.

Другая возможность состоит в том, чтобы установить удаленный конец hts туннеля на finger port (TCP/79) SMTP/DNS хоста и получить список всех пользователей системы. Это позволит получить первое представление о возможных именах учетных записей. Однажды доступ к системе будет получен, либо перебором, либо с помощью эксплойта, например, через

переполнение буфера в `sadmind` и атакующий получит доступ к системе, находящейся за маршрутизатором, к которой в обычных обстоятельствах доступа нет. Получив доступ к командной строке SMTP/DNS хоста, находящегося за маршрутизатором, атакующий может установить дополнительные утилиты для дальнейшего проникновения в сеть. Например, атакующий может следить за соединениями к SMTP/DNS хосту и получать информацию о потенциальных сервисах, запущенных на системах, которые находятся за брандмауэром. Это было бы более скрытным методом разведки, чем прямое сканирование IP адресов в пределах DMZ. Если атакующий идентифицирует хост, на котором запущен SSH сервер (находится за брандмауэром), он может использовать SMTP/DNS хост, чтобы попытаться подключиться к новой цели и использовать любую предполагаемую или взломанную информацию об учетных записях для получения доступа.

3. Скрытый канал поверх ICMP

Скрытый канал поверх ICMP (ICMP-туннель) устанавливает скрытое соединение между двумя компьютерами с помощью пакетов эхо-запроса и эхо-ответа протокола ICMP. Данная техника позволяет, например, полностью туннелировать TCP-трафик через эхо-запросы и ответы (`ping`). Технически туннелирование через скрытый канал ICMP происходит посредством внедрения любых нужных данных в эхо-пакет и его пересылки на удаленный компьютер. Удаленный компьютер отвечает подобным образом, внедряя ответ в другой ICMP-пакет и отсылая этот пакет назад.

Когда кто-нибудь говорит о «пинге порта», он на самом деле говорит об использовании протокола четвертого уровня (вроде TCP или UDP), чтобы выяснить, открыт ли порт. Если кто-либо «пингует» порт 80, это обычно значит, что он посылает на данную систему SYN-пакет по протоколу TCP.

Настоящий `ping` работает через протокол ICMP, который не использует порты вовсе.

На сетевом уровне пакеты маршрутизируются на основании уникального сетевого адреса.

Файрволы работают на разных уровнях, чтобы иметь возможность применять различные критерии для ограничения трафика. Самый низкий уровень работы файрвола – третий. В модели OSI он называется сетевым. В стеке TCP/IP это уровень протокола межсетевого взаимодействия (Internet Protocol). Данный уровень занимается маршрутизированием пакетов к месту их назначения. На третьем уровне файрвол может определить, пришел ли пакет из надежного источника, но не может сказать, что он содержит и с какими другими пакетами связан. Файрволы, которые оперируют на транспортном уровне, знают о пакете немного больше и могут разрешить или запретить доступ на основе более сложных критериев. На прикладном уровне файрволы владеют большим количеством информации о том, что происходит, и могут быть очень избирательны в предоставлении доступа.

ICMP-туннелирование можно использовать для обхода правил файрвола путем обфускации основного трафика. В зависимости от реализации ПО для ICMP-туннелирования этот тип соединения можно отнести и к категории зашифрованных соединений между двумя компьютерами. Без глубокого исследования пакетов или просмотра системных журналов сетевые администраторы не смогут обнаружить этот тип трафика в своей сети.

Чтобы увидеть содержимое ICMP-пакета посылаются запросы на удаленный хост и прослушивается сетевой трафик.

На рисунке 4 показано, как посылаются обычные эхо-запросы протокола ICMP на удаленный хост 192.168.157.1.


```
root@bt: ~ - Shell - Konsole
Session Edit View Bookmarks Settings Help

root@bt:~# ping 192.168.157.1
PING 192.168.157.1 (192.168.157.1) 56(84) bytes of data.
64 bytes from 192.168.157.1: icmp_seq=1 ttl=128 time=4.72 ms
64 bytes from 192.168.157.1: icmp_seq=2 ttl=128 time=0.202 ms
64 bytes from 192.168.157.1: icmp_seq=3 ttl=128 time=0.460 ms
64 bytes from 192.168.157.1: icmp_seq=4 ttl=128 time=0.242 ms
64 bytes from 192.168.157.1: icmp_seq=5 ttl=128 time=0.171 ms
64 bytes from 192.168.157.1: icmp_seq=6 ttl=128 time=0.250 ms
64 bytes from 192.168.157.1: icmp_seq=7 ttl=128 time=0.252 ms
64 bytes from 192.168.157.1: icmp_seq=8 ttl=128 time=0.409 ms
64 bytes from 192.168.157.1: icmp_seq=9 ttl=128 time=0.497 ms
64 bytes from 192.168.157.1: icmp_seq=10 ttl=128 time=0.244 ms
^C
--- 192.168.157.1 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9000ms
rtt min/avg/max/mdev = 0.171/0.745/4.728/1.332 ms
root@bt:~#
```

Рисунок 4 – Эхо – запросы протокола ICMP

Захватив трафик с помощью Wireshark, мы увидим следующее (Рис.5):

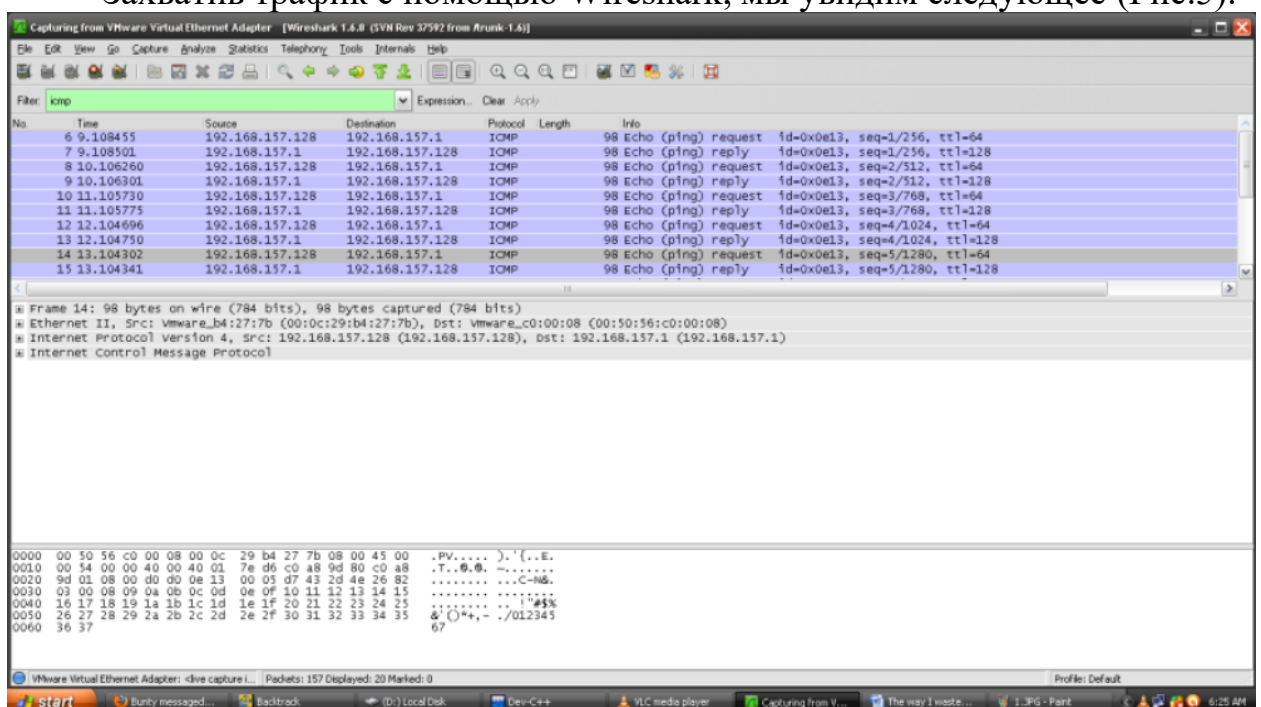


Рисунок 5 – Захват трафика при помощи Wireshark

Поскольку мы использовали стандартную утилиту ping без опций, мы послали несколько последовательных ICMP-запросов к хосту. Это несколько

затруднит нам анализ трафика. Поэтому давайте пошлем одиночный ICMP-пакет без нагрузки.

Для этой цели можно использовать следующую команду.

```
ping -c 1 -s 0
```

```
10 packets transmitted, 10 received, 0% packet loss, time 9000ms
rtt min/avg/max/mdev = 0.171/0.745/4.728/1.332 ms
root@bt:~# ping -c 1 -s 0 192.168.157.1
PING 192.168.157.1 (192.168.157.1) 0(28) bytes of data.
8 bytes from 192.168.157.1: icmp_seq=1 ttl=128
--- 192.168.157.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms

root@bt:~#
```

BT4..zip

Теперь анализировать трафик будет проще.

В сниффере на удаленном хосте мы можем видеть, что получено 42 байта данных.

```
00 50 56 c0 00 08 00 0c 29 b4 27 7b 08 00 45 00 .PV.....).'{..E.
00 1c 00 00 40 00 40 01 7f 0e c0 a8 9d 80 c0 a8 .....@. ....
9d 01 08 00 e8 eb 0f 13 00 01 ..... ..
```

Типичная структура ICMP-пакета представлена на рисунке 5.

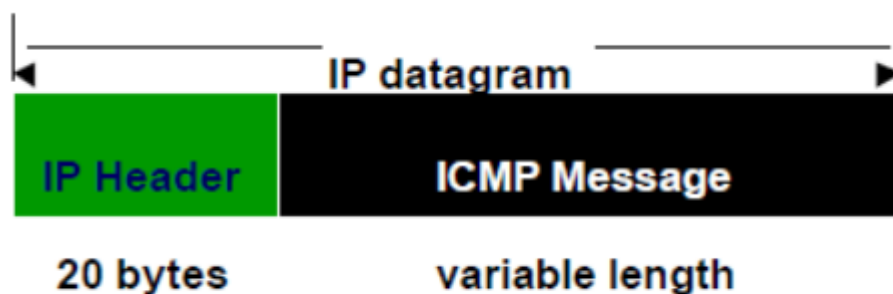
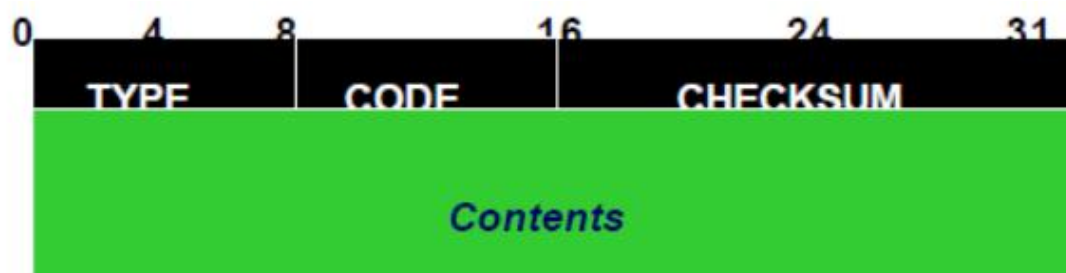


Рисунок 5 – Типичная структура ICMP - пакета

Типичная структура ICMP-заголовка (Рис. 6):



- **TYPE:** Type of ICMP message
- **CODE:** Used by some types to indicate a specific condition
- **CHECKSUM:** Checksum over full message
- **Contents** depend on TYPE and CODE

Рисунок 6 – Типичная структура ICMP-заголовка

Анализируя те 42 байта данных, мы можем заключить, что первые 14 байтов представляют собой Ethernet-заголовок.

```
00 50 56 c0 00 08 00 0c 29 b4 27 7b 08 00 45 00 .PV.....).'{..E.
00 1c 00 00 40 00 40 01 7f 0e c0 a8 9d 80 c0 a8 ....@.@. ....
9d 01 08 00 e8 eb 0f 13 00 01 ..... ..
```

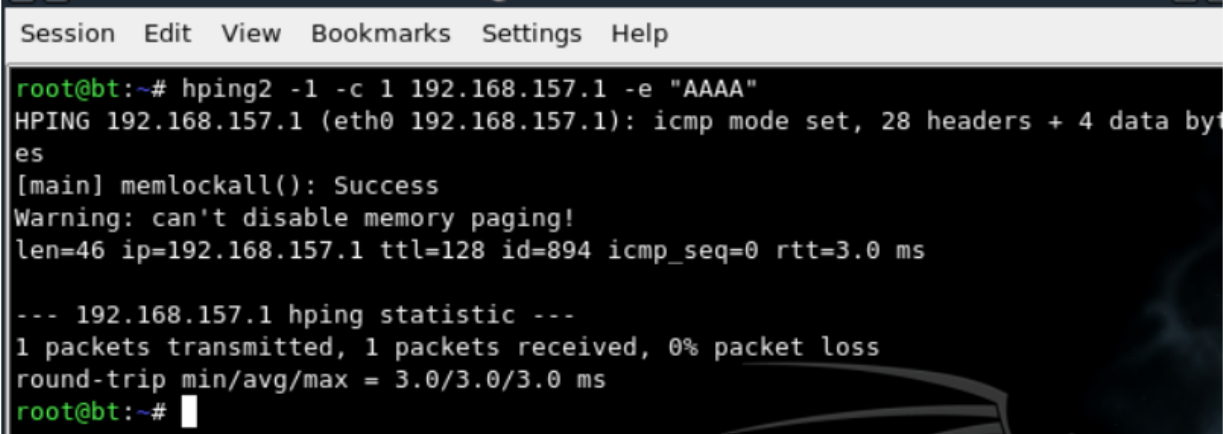
Структура IP-заголовка, представленная более подробно.

```
45      VERSION IPV4
00      Different Services
00 1c   Total length 28
00 00   Identification 0x00000
40 01   Dont Fragment offset
40      Time to live
01      ICMP version 1
7f 0e   Header Check sum
c0 a8 9d 80 --- Source Ip   192.168.157.128
c0 a8 9d 01 -- Destination IP   192.168.157.1
```

Утилита ping, присутствующая в обычных Linux-системах, позволяет нам контролировать количество пакетов, их размер, но не позволяет сформировать эхо-запрос специального вида. Поскольку главной нашей целью является манипулирование данными в содержимом ICMP-пакета, использование стандартной утилиты ping – не лучший вариант.

HPING2

Hping – свободный генератор и анализатор пакетов для TCP/IP протоколов, распространяемый Salvatore Sanfilippo (также известным как Antirez). С помощью hping мы также можем манипулировать порцией данных одиночного ICMP-пакета. Используя hping, снова посылается одиночный ICMP-пакет.



```
Session Edit View Bookmarks Settings Help
root@bt:~# hping2 -l -c 1 192.168.157.1 -e "AAAA"
HPING 192.168.157.1 (eth0 192.168.157.1): icmp mode set, 28 headers + 4 data bytes
[main] memlockall(): Success
Warning: can't disable memory paging!
len=46 ip=192.168.157.1 ttl=128 id=894 icmp_seq=0 rtt=3.0 ms

--- 192.168.157.1 hping statistic ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 3.0/3.0/3.0 ms
root@bt:~#
```

Рисунок 7 – Утилита hping2

На рис.7 можно заметить, что мы добавили в пакет 4 символа “А” и послали его на адрес 192.168.157.1.

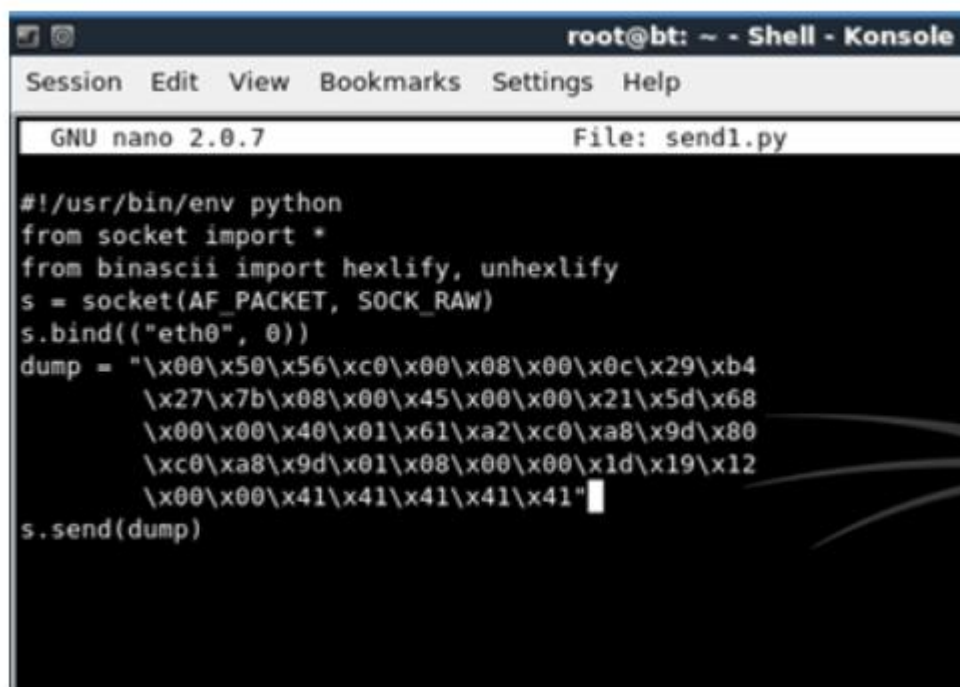
Проанализируем трафик:

00	0c	29	b4	27	7b	00	50	56	c0	00	08	08	00	45	00	..).	{.	P	V.....E.
00	20	03	7e	00	00	80	01	7b	8c	c0	a8	9d	01	c0	a8	..~....	{.....		
9d	80	00	00	4b	6a	32	13	00	00	41	41	41	41			Kj2.	..AAAA		

Рисунок – 8 Анализ захваченного трафика

Теперь мы можем видеть, что был захват 46(42+4) байтов данных. На рис. 8 легко различить нашу нагрузку, состоящую из подсвеченных "41". Это hex-представление символа “А”.

В вышеприведенных примерах мы использовали для отправки ICMP эхо-запросов на любой хост стандартную утилиту ping и утилиту hping. Теперь мы используем Python для отсылки самодельного ICMP-пакета.

A screenshot of a terminal window titled "root@bt: ~ - Shell - Konsole". The window shows the GNU nano 2.0.7 text editor editing a file named "send1.py". The script is a Python program that uses the socket module to create a raw socket, bind it to the interface "eth0", and send a packet. The packet data is stored in a variable named "dump" and consists of a series of hexadecimal bytes representing an ICMP Echo request. The script ends with "s.send(dump)".

```
#!/usr/bin/env python
from socket import *
from binascii import hexlify, unhexlify
s = socket(AF_PACKET, SOCK_RAW)
s.bind(("eth0", 0))
dump = "\x00\x50\x56\xc0\x00\x08\x00\x0c\x29\xb4
        \x27\x7b\x08\x00\x45\x00\x00\x21\x5d\x68
        \x00\x00\x40\x01\x61\xa2\xc0\xa8\x9d\x80
        \xc0\xa8\x9d\x01\x08\x00\x00\x1d\x19\x12
        \x00\x00\x41\x41\x41\x41\x41"
s.send(dump)
```

Рисунок 9 – Использование Python для отсылки самодельного ICMP-пакета

В данном случае мы формируем не только заголовок ICMP и начинку пакета, но и заголовки протоколов Ethernet и IP.

Переменная “dump” в вышеприведенном скрипте содержит всю датаграмму целиком, включая Ethernet-заголовок, IP-заголовок, ICMP-заголовок и начинку. Здесь выступают четыре символа “A” (\x41\x 41\x 41\x 41).

Из сказанного выше ясно, что злоумышленник может легко послать произвольные данные на произвольный хост путем внедрения данных в эхо-пакет. Теперь осталось злоумышленнику запустить на удаленном хосте демон, который умеет отвечать подобным образом, внедряя ответ в другой ICMP-пакет и посылая пакет назад.

Задание к практической работе

В практической работе необходимо настроить:

- сервер на ОС Windows;

- клиент на основе Kali-Linux или др. дистрибутивов, в составе которых есть утилита hping;
- анализатор пакетов для TCP/IP протоколов Wireshark.

Результаты построения оформить в виде отчета со следующими требованиями:

- IP-адрес сети должно соответствовать схеме 192.168.<учащегося>. 1/24;
- перед выполнением ping необходимо сначала показать IP-адрес интерфейса, с которого производится команда;
- нагрузка (добавленные данные) должна содержать <Фамилию>;
- в случае использования современных версий необходимо представить исходный рабочий код сервера и клиента.

Для повышения оценки (оценок) факультативом является настройка ICMP туннеля между двумя Linux-системами.

Составьте отчет о выполнении практической работы.

Включите в него копии экрана пошагового выполнения задания и ответы на вопросы практической работы.